Informe de testing NutriVida

Cobertura y resultados de la Evaluación Parcial 2

El proceso de verificación de NutriVida V2 terminó sin fallos en las suites de pruebas, la revisión estática y el build. Este informe distingue pruebas aprobadas de cobertura de código y documenta los casos ejecutados, las herramientas y los límites de los resultados.

Ejecución	Resultado
Vitest y Testing Library	74 pruebas aprobadas en 10 archivos.
Jasmine y Karma	94 ejecuciones aprobadas en Chrome Headless.
ESLint	Sin errores.
Build Vite	Compilación de producción correcta.
Fecha	18 de octubre de 2026
Autores del proyecto	Antonia Muñoz y Felipe Araya

Cobertura global medida

Métrica	Vitest V8	Jasmine Istanbul
Sentencias	89.86 %	89.96 %
Ramas	80.58 %	80.69 %
Funciones	90.46 %	90.46 %
Líneas	90.75 %	90.87 %

La suite Jasmine ejecuta los 74 casos compartidos y 20 casos nativos adicionales. No se deben sumar 74 y 94 como si fueran 168 casos distintos: existe reutilización intencional de escenarios.

Los porcentajes difieren ligeramente por los proveedores V8 e Istanbul y sus mapas de código. Ninguno representa cobertura del 100 % ni demuestra por sí solo ausencia de defectos.

Entorno y proceso de pruebas

Herramienta	Versión verificada
Node.js	v24.19.0
react	19.3.1
bootstrap	5.3.8
vite	8.3.4
vitest	5.0.5
jasmine-core	7.0.2
karma	6.4.4
karma-coverage	2.2.1

Configuración y ejecución

Vitest usa jsdom y Testing Library para renderizar componentes y simular eventos. Karma usa Jasmine en Chrome Headless con un perfil de pruebas dedicado. El bundle de pruebas se prepara con esbuild; Babel e Istanbul instrumentan únicamente el código de la aplicación.

Los casos compartidos se registran en suites Jasmine aisladas por archivo. Una capa pequeña adapta las aserciones DOM utilizadas por las pruebas existentes; el motor que ejecuta test:jasmine es Jasmine, no Vitest. Los veinte casos J01 a J20 están escritos para demostrar explícitamente Jasmine, incluyendo un spy nativo.

Comandos reproducibles

```
npm ci
npm run test:coverage
npm run test:jasmine
npm run lint
npm run build
npm run verify
```

Se requiere Node.js 24 y Chrome o Edge instalado. Puede indicarse CHROME_BIN si el navegador está en otra ruta. La alternativa con pnpm está descrita en el README.

Cada prueba prepara datos ficticios, realiza una acción y comprueba el resultado. localStorage, sessionStorage, el historial de navegación y los componentes montados se limpian entre casos. No se opera sobre los datos del navegador con el que el usuario usa la aplicación.

Alcance y ejemplos de pruebas

Caso	Concepto	Verificación
J01	Propiedades	Renderiza una tarjeta con título y descripción recibidos por props y comprueba su enlace.
J02	Evento y renderizado condicional	El clic añade el servicio al carrito y muestra la confirmación solamente después de la acción.
J03	Estado y DOM	Cambiar la búsqueda reduce el catálogo a los servicios coincidentes.
J04	Compra aprobada	Guarda una orden por 25.000 CLP, estado Pagada, y vacía el carrito.
J05	Compra rechazada	Guarda el rechazo y conserva el carrito para reintentar.
J06	Idempotencia	Repetir el identificador de una solicitud devuelve la misma orden sin duplicarla.
J07	Spy y error simulado	spyOn intercepta el almacenamiento para comprobar que un fallo no se anuncia como éxito.
J08	CRUD personal	Crea un registro de progreso y elimina solo el registro de la cuenta activa.
J09	Validación	Rechaza una fecha futura y un peso fuera del intervalo definido.
J10	CRUD administrativo	Crea, edita y elimina una categoría y comprueba sus datos persistidos.

Otros escenarios incluidos

Los casos compartidos cubren autenticación, roles protegidos, registro, perfil, reservas y horarios, cancelación propia, filtros, persistencia, estados de citas, precio histórico, edición de servicios y usuarios, compras y privacidad entre cuentas. También se prueban los artículos, las categorías, el progreso, los planes asignados y los enlaces HTML migrados.

Se incluyen tanto pruebas unitarias de funciones como pruebas de integración de componentes, navegación y almacenamiento. Las pruebas de DOM no se presentan como una prueba integral de despliegue o una revisión visual completa.

Análisis de cobertura y resultados

Módulo	Líneas Vitest	Ramas Vitest
commerce.js	89.15 %	89.87 %
clinical.js	95.23 %	87.5 %
booking.js	83.72 %	80.39 %
admin.js	93.57 %	77.77 %
Nosotros.jsx	100 %	55 %
Plan.jsx	100 %	75 %
Business.jsx	87.09 %	55.55 %

Interpretación

La cobertura de líneas global supera el 90 %. La cobertura de ramas es menor: ejecutar una línea no significa haber recorrido todas sus decisiones. Los porcentajes de cada módulo se conservan en los JSON y en los reportes HTML para revisión del docente.

Las principales áreas para ampliar pruebas son las combinaciones de usuarios y campos opcionales en Nosotros, las listas vacías y errores del módulo administrativo de categorías, los errores de persistencia en operaciones secundarias y algunas condiciones de horarios del día actual.

Mocks y manejo de errores

J07 usa spyOn de Jasmine para lanzar un error controlado al escribir progreso y comprueba que no se registre un guardado exitoso. En las pruebas compartidas se simula un fallo de almacenamiento al reservar. Los spies se restauran y los datos se aíslan para no contaminar los escenarios posteriores.

La compra rechazada es un resultado de negocio simulado, no un fallo de una entidad bancaria real. Se verifica que conserva el carrito y permite volver al checkout.

Evidencia disponible

docs/evidencias conserva resumen.json, cobertura-vitest.json, cobertura-jasmine.json y los logs de Vitest, Jasmine, ESLint y build. coverage/vitest y coverage/jasmine contienen los reportes HTML y LCOV regenerables. El comprimido de entrega incluye los reportes de cobertura de esta ejecución.

Reproducción aceptación y límites

Recorrido manual recomendado

Paso	Comprobación
1	Registrar un paciente y editar su perfil; comprobar persistencia al salir y volver a ingresar.
2	Filtrar categorías y ofertas; añadir dos servicios y modificar la cantidad en el carrito.
3	Completar un pago aprobado y otro rechazado; abrir sus comprobantes y reintentar.
4	Reservar una cita hábil, verla en ambas vistas y cancelarla desde la cuenta del paciente.
5	Entrar como administrador, editar servicios, categorías y usuarios y consultar reportes.
6	Entrar como médico, comprobar la agenda asignada y publicar un plan para el paciente.
7	Revisar menú, formularios y tablas en móvil, tableta y escritorio.

Este recorrido es una guía para la defensa y la aceptación visual; no se marca como ejecutado manualmente en todos los dispositivos. Las suites automatizadas sí fueron ejecutadas en las condiciones descritas.

Observaciones técnicas

npm audit sin dependencias de desarrollo informa cero vulnerabilidades en el árbol de producción de esta instalación. El árbol completo informa nueve avisos en dependencias transitivas del entorno de testing legado. No se aplicaron cambios mayores incompatibles solo para eliminar avisos; Karma permanece por exigencia de la pauta. Se recomienda usar el runner únicamente en el entorno local de pruebas, como está configurado.

Los datos y permisos son simulados en el navegador. No hay backend real, cobro bancario, boleta tributaria, pruebas de carga, auditoría de seguridad de producción ni validación clínica de las indicaciones. La nota y la defensa individual no se pueden garantizar mediante estas pruebas.

Referencias para reproducir

Documentación de Jasmine: https://jasmine.github.io/tutorials/configuring_jasmine
Configuración de Karma: https://karma-runner.github.io/6.4/config/browsers.html
Anexo e instrumento de evaluación DSY1104 aportados por el equipo.
Código fuente y evidencias de NutriVida V2.